Lab 4 AVCV 2026/2027: Fire-risk classification from satellite images

CUSTOM and FINE-TUNING networks on the FireRisk subset

Daniel Truzman (100475161) César Álvarez-Cascos (100475191) • CodaBench team `100475161_100475191`

1. INTRODUCTION

Context and objective. Mapping wildfire risk over large areas from satellite imagery can support the planning of prevention efforts. We address it as *image classification*: given one RGB satellite image, predict one of the seven FireRisk classes (Water, Very Low, Low, Moderate, High, Very High, Non-burnable). The task is neither fire detection nor temporal forecasting: the label describes the risk of the area, not the presence of fire. Two properties make it hard: strong class imbalance, and risk levels that are ordinal and visually similar, so that neighbouring classes (e.g. Low/Moderate/High) are easily confused.

Dataset. We use a subset of FireRisk [1] with 5,861 RGB images of 320×320 pixels, split in a stratified way into 3,516 training, 1,172 validation and 1,173 test images (test labels are hidden). The training set contains 86 Water, 1,088 Very Low, 535 Low, 431 Moderate, 315 High, 163 Very High and 898 Non-burnable images (Table 2); validation has the same proportions, so the rarest class (Water, 2.4%) is 13 times less frequent than the most common one (Very Low, 31%). The original images are 320×320, whereas both final networks receive 224×224 crops.

Scope. We develop two systems: CUSTOM, a VGG-style CNN trained from scratch and selected through a staged random hyperparameter search, and FINE-TUNING, which adapts ImageNet-pretrained torchvision models (ConvNeXt-T, Swin-T, ResNet-50), with Swin-T selected. The main questions studied are network width and depth, number of epochs, class weighting, augmentation and image mixing, ensembles, and retraining on train+val.

2. METHODOLOGY

2.1. Preprocessing and data augmentation

Pipeline. Images are read as float tensors in $[0, 1]$. For training, a random 224×224 crop is taken from the 320×320 image on the CPU; validation and test images use a deterministic 224×224 centre crop. All images are normalised with the ImageNet mean and standard deviation, which the pretrained weights require and which we keep for CUSTOM for a common pipeline. Training batches are then augmented on the GPU with random horizontal and vertical flips, random 90° rotations and random gamma, brightness, contrast and saturation changes, with two presets: *light* (0.1/0.05/0.05/0.05) and *strong* (0.2/0.15/0.15/0.15). Validation and test receive no augmentation and no test-time augmentation (TTA).

Justification. Satellite images have no canonical orientation, so flips and 90° rotations create new samples without changing the label, while colour changes are kept mild to preserve vegetation cues. Random crops add translation variety and reduce the input cost. The random search selected *strong* for the best CUSTOM model, while the selected Swin-T uses *light*. Other options were tested on validation and discarded: mixup [2] (α 0.2/0.4) was not kept; CutMix [3], standard or mixing only the five risk classes, gave a last-10-epoch mean val F1 of 0.483 and 0.470 versus 0.486 without it; TTA with 8 flips/rotations lowered val F1 by about 0.006; and training at 288 while evaluating on the full 320 image gained at most +0.01, within seed noise ($\pm$0.008).

2.2. System design: CUSTOM network

Architecture. The CUSTOM network (Table 4) is a VGG-style stack [4] built only from primitive layers. Each of its four blocks applies twice the sequence Conv 3×3 (stride 1, padding 1)–BatchNorm [5]–ReLU and ends with 2×2 max-pooling, so the channel width grows as 48–96–192–384 while the 224×224 input shrinks to a 14×14 map. Global average pooling (GAP) [6], computed as a spatial mean, yields a 384-dimensional descriptor, and the classifier FC(384,256)–ReLU–Dropout(0.3)–FC(256,7) returns seven raw logits (the softmax is part of the loss). Weights use Kaiming initialisation; the model has 2,739,447 parameters.

Design decisions. The provided reference has three blocks of a single unpadded 5×5 convolution (6–16–32 channels), no normalisation, a Flatten head and SGD with StepLR. Each change below was kept only if it raised validation macro-F1. (i) GAP replaces Flatten: with Flatten the first FC layer held about 99% of the parameters, and Flatten heads scored below GAP heads in the first search phase. (ii) Two stacked 3×3 convolutions see a 5×5 field with fewer weights and one more non-linearity [4], and BatchNorm stabilises training from scratch. (iii) Widening to 48 base channels with two convolutions per block added about +0.03 macro-F1, whereas 64 base channels, a fifth block or larger crops gave no gain beyond seed noise (Table 1). (iv) The mean replaces `AdaptiveAvgPool2d`, whose CUDA backward pass is not deterministic (Section 2.4). Regularisation combines dropout in the head, decoupled weight decay and data augmentation (Section 2.1).

2.3. System design: FINE-TUNING network

Pretrained model. Instead of the provided AlexNet baseline [7], we compared three modern backbones of similar size (24–28 M parameters) from different families, all with torchvision ImageNet-1k weights [8]: ConvNeXt-Tiny [9] (`IMAGENET1K_V1`), EfficientNetV2-S [10] (`IMAGENET1K_V1`) and Swin-T [11] (`Swin_T_Weights.IMAGENET1K_V1`). EfficientNetV2-S failed our compatibility test (non-finite loss under fp16 mixed precision) and was replaced by the reserve ResNet-50 [12] (`IMAGENET1K_V2`). In every model the final layer was replaced by a freshly initialised `Linear(in_features, 7)` (768→7 for Swin-T), every `AdaptiveAvgPool2d(1)` was replaced by an equivalent mean over H, W to obtain a deterministic CUDA backward pass, and the stochastic-depth rate [13] was reset to the sampled value.

Adaptation strategy. No layers were frozen: backbone and head were fine-tuned jointly. Inputs were resized to the pretraining resolution (224×224) and normalised with the ImageNet mean and standard deviation. The optimiser [14] used two parameter groups: the backbone learning rate (log-uniform search over $[2 \cdot 10^{-5}, 3 \cdot 10^{-4}]$ for ConvNeXt/Swin and $[5 \cdot 10^{-5}, 8 \cdot 10^{-4}]$ for ResNet) and a head rate multiplied by `head_mult`$\in \{1, 10\}$, in the spirit of discriminative fine-tuning [15]; biases, normalisation layers, layer-scale parameters and relative-position tables were excluded from weight decay. The selected configuration, Swin-T with backbone rate $2.37 \cdot 10^{-5}$, `head_mult`$= 1$ and drop path 0.2, reached the highest validation macro-F1 (0.546 at epoch 26/30; 0.538 averaged over the last five epochs), ahead of the best ResNet-50 (0.539) and ConvNeXt-T (0.505) runs (Table 1). It was also the most stable: its training F1 was 0.77, whereas the ResNet-50 run reached 0.99, a sign of strong overfitting.

2.4. Training protocol and model selection

Configuration. Both models minimise cross-entropy with class weights $w_c \propto 1/\sqrt{n_c}$ (mean 1), without label smoothing or mixup [2]. Both use AdamW [14], a linear warm-up followed by a per-epoch cosine decay to zero [16], and fp16 mixed precision. CUSTOM: learning rate 3.17×10^{-4}, weight decay 0.0464, batch 128, 1 warm-up epoch, 60 epochs, strong augmentation. FT (Swin-T): backbone learning rate 2.37×10^{-5} (head multiplier 1), weight decay 0.0395 (none on biases, normalisation layers and position tables), batch 32, 2 warm-up epochs, 30 epochs, drop path 0.2, light augmentation. Every run uses seed 42 and the full training split (3,516 images, no subsampling) on a Kaggle Tesla T4 (16 GB) with PyTorch 2.11 [17].

Hyperparameter search. CUSTOM hyperparameters come from a staged random search [18] with median-stopping pruning [19]. Phase 1 drew 24 architecture and optimiser trials (15 epochs, pruned at epochs 4 and 8); phase 2 ran 16 trials of secondary hyperparameters around the best one; in phase 3 the three finalists, trained for 40 epochs, plateaued at 0.45–0.465 with train ≈ validation F1 (underfitting). Phase 4 therefore tested targeted capacity changes (the 48-channel widening), and phase 5 ran 12 deterministic runs in which only the 60-epoch budget gave a clear gain (seed noise ±0.008). A deeper search of 121 trials (balanced design plus an evolutionary phase) reached 0.524 on validation but only 0.47 on the public test and was discarded. For FT, 10 configurations per network were run for 10 epochs (pruned at epoch 4 against the same network) and the top two per network were retrained for 30 epochs.

Selection and reproducibility. Validation macro-F1 is computed after every epoch and the best epoch's weights are kept: epoch 36 (0.502) for CUSTOM and epoch 26 (0.546) for FT. No early stopping was used in the final runs (pruning exists only in the search). Training is fully deterministic: `torch.use_deterministic_algorithms(True)`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, deterministic cuDNN without benchmarking, and fixed Python, NumPy and PyTorch seeds, also for the four DataLoader workers (generator and `worker_init_fn`). A self-test that trains one epoch twice produced bit-identical weights. The validation split is used only for selection. Because it was reused many times, its scores are optimistic: we observed a validation-to-public gap of about 0.06 (Table 3).

3. RESULTS AND ERROR ANALYSIS

Evaluation metrics. The main metric is the macro-F1, the unweighted mean of the seven per-class F1 scores, so the rare classes (Water, Very High; Table 2) weigh as much as the frequent ones; per-class scores are given in Table 2. Training F1 is only a diagnostic, every decision used the validation F1 (1,172 images), and the public CodaBench score (a fixed, undisclosed subset of the test set) was never used for selection.

Experimental comparison. Table 1 summarises the most informative runs. The reference CUSTOM notebook reached about 0.36 (StepLR divided the learning rate by 10 every 7 epochs) and our first GAP version 0.440 with training F1 ≈ 0.42, i.e. underfitting. Phases 1–3 of the search plateaued at 0.45–0.465 with train ≈ val, and inverse-frequency weights hurt (last-10 mean 0.431 vs 0.453). The decisive change was capacity: 48 channels and two convolutions per block raised the last-10 mean to 0.483, and 60 epochs gave the selected n07_60ep (0.502 at epoch 36, last-10 0.489). Further changes (64 channels, a fifth block, a larger crop, CutMix, 80 epochs, a probability ensemble (0.492) and retraining on train+val) brought no gain; per-class bias tuning gave +0.022 in-sample but ≈0 under 2-fold CV, and seeds alone spread results by ±0.008. For FINE-TUNING, every modern backbone beat the provided AlexNet recipe (0.452); Swin-T (swin_p2_1, 0.546) was preferred to ResNet-50 (0.539, training F1 0.99).

Learning curves and confusion matrices. In Figure 2 the CUSTOM training and validation losses decrease

together and the validation macro-F1 saturates around 0.49–0.50 from epoch 30, with the (augmented) training F1 at the same level (last-10 mean 0.524): the model does not overfit. The selected FT model, instead, reaches a training F1 of 0.77 against 0.546 on validation. The validation confusion matrices (Figure 3, row-normalised %, rows = true class) show where the remaining errors lie [TODO: describe once the matrices are generated].

Visual error analysis. Water (F1 0.847) and Non-burnable (0.796) are recognised reliably, whereas Low (0.221), Moderate (0.322) and High (0.333) are the weakest CUSTOM classes, and Low (0.17–0.33), Moderate (0.30–0.39) and Very High (0.29–0.43) the weakest for every FT model. The weak classes are the intermediate, ordinal risk levels, which suggests confusions between adjacent levels [TODO: confirm with Figures 3 and 4 and describe the examples]. Even on the training set, evaluated without augmentation, the n07/n09 checkpoints reach only 0.53/0.59 (Low 0.30), which points to limited visual signal and label noise rather than lack of capacity. CUSTOM is small (2.74 M parameters), fast and stable but saturates near 0.50; FT gains about 0.04 on validation from ImageNet features, at a higher compute cost and with more overfitting.

Competition feedback. Table 3 lists the public scores: 0.49 for the best CUSTOM entry (p4_09_B, submission 1036062) and 0.48 for the selected FT model. Every model, ensembles included, loses about 0.06 from validation to public, because the same validation images were reused over hundreds of trials, the public subset is noisy and the intermediate classes are ambiguous; the predicted test class proportions match validation, so no distribution shift is apparent. The deep-search model F3_B_038 illustrates this: best validation score (0.524), only 0.47 on public.

4. CONCLUSIONS

For the CUSTOM network, the changes with the clearest support were replacing the Flatten head with global average pooling, widening to 48 channels with two convolutions per block (+0.03 macro-F1) and a 60-epoch cosine schedule, which reached 0.502 on validation. Inverse-frequency class weights, further width or depth, larger crops, CutMix, 80 epochs, ensembles, per-class bias tuning and retraining on train+val (0.494 public) brought no reliable gain. For FT, ImageNet pretraining itself was decisive: it reaches 0.52–0.54 validation macro-F1 in about 10 epochs, against 0.46 for the scratch model after 15, and the best Swin-T (0.546) used a small backbone learning rate (2.37×10^{-5}) with an unscaled head. However, the public scores are close (0.49 CUSTOM, 0.48 FT), so the gap between categories largely vanishes on the public subset.

Every model loses about 0.06 from validation to public. We attribute this to selection bias from reusing the same 1,172 validation images, a noisy public subset, and ambiguous ordinal intermediate classes: even on its training set the CUSTOM model only reaches 0.53 macro-F1 in eval mode, with Low at roughly 0.2–0.3, which suggests limited visual signal at this resolution. The original FireRisk benchmark reports only ∼61–65% accuracy using the full ∼91k images [1]. Realistic next steps are stratified k-fold over train+val with fold ensembling for honest validation, ordinal-aware losses, remote-sensing pretraining and higher input resolution for FT.

Individual contributions. **Daniel Truzman (100475161):** GPU data pipeline and augmentation with AMP optimisation, notebook cleanup, phases 3–4 of the CUSTOM search on Colab and Kaggle (48-channel, two-conv widening; class-weight tests), phase 5 (12 strictly deterministic Kaggle runs, 60 epochs, selecting `n07_60ep`), determinism setup, CutMix, ensemble, bias-tuning and train+val experiments, and the CodaBench submissions. [TODO: confirm (GPU pipeline/AMP: git log shows a "GPU Performance" commit by César)]
César Álvarez-Cascos (100475191): Parts 1–2 of the notebook (dataset and transforms), the two-machine random-search infrastructure (phases 1–2), the deep hyperparameter search (121 trials) and the fine-tuning search over ConvNeXt-T, Swin-T and ResNet-50, which selected Swin-T. [TODO: confirm]

Table 1: Main experiments: validation macro-F1 at the best epoch (L10: mean of the last 10 epochs). Reference and selected models in bold.

Cat.	Change	Objective	Budget	Val F1
CUSTOM	**Reference notebook (StepLR)**	Baseline	[TODO:] ep	≈0.36
CUSTOM	GAP instead of Flatten (first version)	Fewer FC parameters; train F1 ≈0.42	[TODO:] ep	0.440
CUSTOM	Phases 1–3 random search	Architecture, optimiser, regularisation	15 ep (finalists 40)	0.45–0.465
CUSTOM	Inverse-frequency class weights	Help minority classes	[TODO:] ep	L10 0.431 vs 0.453
CUSTOM	48 ch, 2 convs/block (p4_09_B)	More capacity (underfitting)	40 ep	0.496 (L10 0.483)
CUSTOM	64 ch; 5 blocks; crop 288/eval 320	More capacity / resolution	40 ep	no gain
CUSTOM	**60 epochs (n07_60ep)**	Longer schedule	60 ep	**0.502** (L10 0.489)
CUSTOM	CutMix (standard; risk-only)	Regularisation	[TODO:]; 80 ep	L10 0.483; 0.470 vs 0.486
CUSTOM	Deep search (F3_B_038)	Joint search, 121 trials	80 ep, input 320	0.524
FT	**AlexNet (provided recipe)**	Baseline	5 ep	0.452
FT	ConvNeXt-T (cnx_p2_0)	Modern CNN backbone	30 ep	0.505
FT	ResNet-50 (r50_p2_0)	Residual backbone (train F1 0.99)	30 ep	0.539
FT	**Swin-T (swin_p2_1)**	Transformer backbone	30 ep	**0.546**

Table 2: Class counts and per-class validation F1 of the selected models. [TODO: P/R]

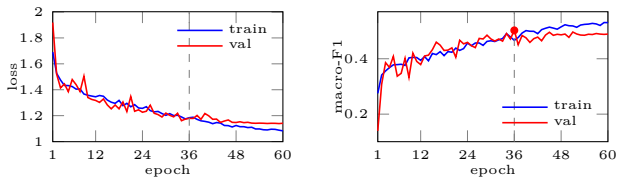
Class	Train	Val	CUSTOM	FT
0 Water	86	29	0.847	[TODO:]
1 Very Low	1,088	363	0.598	
2 Low	535	178	0.221	
3 Moderate	431	144	0.322	
4 High	315	105	0.333	
5 Very High	163	54	0.400	
6 Non-burnable	898	299	0.796	
Total / macro	3,516	1,172	0.502	0.546

Table 3: Public CodaBench macro-F1 (fixed undisclosed test subset). Also: F1_B_037 0.47, swin_p2_0 0.48, r50_p2_0 0.47.

ID	Date	Models (custom / ft)	CUSTOM	FT
1036062	05 Nov	p4_09_B / placeholder	0.49	–
[TODO:]	[TODO:]	n07 (train+val) / [TODO:]	0.494	[TODO:]
[TODO:]	[TODO:]	F3_B_038 / AlexNet	0.47	[TODO:]
[TODO:]	[TODO:]	[TODO:] / swin_p2_1	[TODO:]	0.48
[TODO:]	[TODO:]	[TODO:] / FT ensemble (3)	[TODO:]	0.48

Table 4: Final CUSTOM architecture.

Stage	Layers	Output
Input	RGB crop	3 × 224 × 224
Blocks 1–4	[Conv3×3-BN-ReLU]×2, MaxPool 2×2 (spatial size 112/56/28/14)	48/96/192/384 ch 384 × 14 × 14
GAP	spatial mean	384
Head	FC 384→256, ReLU, Dropout 0.3, FC 256→7	7
2,739,447 parameters; Kaiming initialisation.		



(a) Loss **(b)** Macro-F1

Figure 2: Train (augmented) and validation curves, CUSTOM n07_60ep; dashed: selected epoch 36. [TODO: FT curves]

[TODO: CUSTOM CM]

[TODO: FT CM]

(a) CUSTOM (n07_60ep) **(b)** FT (swin_p2_1)

Figure 3: Validation confusion matrices, row-normalised (%); rows = true, columns = predicted class.

[TODO: one row of misclassified validation images, each with true / predicted label]

Figure 4: Representative validation errors of the selected models (true → predicted class).

REFERENCES

[1] Shuchang Shen, Sachith Seneviratne, Xinye Wanyan, et al. FireRisk: A remote sensing dataset for fire risk assessment with benchmarks using supervised and self-supervised learning. In *DICTA*, 2023.

[2] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[3] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, et al. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *ICLR*, 2014.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[8] Olga Russakovsky, Jia Deng, Hao Su, et al. ImageNet large scale visual recognition challenge. *IJCV*, 2015.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[10] Mingxing Tan and Quoc V. Le. EfficientNetV2: Smaller models and faster training. In *ICML*, 2021.

[11] Ze Liu, Yutong Lin, Yue Cao, et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[13] Gao Huang, Yu Sun, Zhuang Liu, et al. Deep networks with stochastic depth. In *ECCV*, 2016.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[15] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *ACL*, 2018.

[16] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[17] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[18] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *JMLR*, 2012.

[19] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, et al. Google vizier: A service for black-box optimization. In *KDD*, 2017.